

PHP Form Handling

Afrizal Doewes

Introduction

- PHP is commonly used for processing form data submitted by users.
- Form handling involves collecting user input from HTML forms and processing it in PHP scripts.
- The PHP superglobals `$_GET` and `$_POST` are used to collect form-data.

HTML Form Structure

- HTML <form> tag is used to create a form.
- Input elements like text fields, checkboxes, radio buttons, etc., are placed within the form.
- Each input element has a “**name**” attribute to identify it in PHP.

Example

```
<form action="welcome.php" method="GET">\n\n    Name: <input type="text" name="name"><br>\n    E-mail: <input type="text" name="email"><br>\n    <input type="submit">\n\n</form>
```

Sending Data to PHP

- When the form is submitted, data is sent to the server using either the **GET** or **POST** method.
- Both GET and POST methods create arrays of key/value pairs, accessible via `$_GET` and `$_POST` superglobals.
- Developers typically prefer **POST** for form data submission.

GET vs POST methods

- When to use **GET**:
 - Visible in URL, limited to about 2000 characters.
 - Useful for non-sensitive data and bookmarkable pages.
 - Never use for passwords or sensitive information.
- When to use **POST**:
 - Data is embedded in the request body, not visible in URL.
 - No size limits, suitable for sensitive data.
 - Supports advanced functionality like file uploads.
 - Not bookmarkable due to hidden data.

Accessing Form Data in PHP

- PHP uses `$_GET[]` and `$_POST[]` superglobal arrays to access form data based on the method used.
- Example: `$username = $_POST['username'];`

```
<?php
    $username = $_POST['username'];
?>
```

Form Validation

- Validate user input to ensure data integrity and security.
- Check for required fields, correct data types, and format.
- Use PHP functions like `filter_var()`, regular expressions, or custom validation functions.

Example

```
if ( $_SERVER['REQUEST_METHOD'] == 'POST' ) {  
    if ( empty( $_POST['name'] ) ) {  
        $nameErr = "name is required";  
    } else {  
        $name = $_POST['name'];  
        if ( !preg_match("/^[a-zA-Z-' ]*$/", $name) ) {  
            $nameErr = "Only letters and white space allowed";  
        }  
    }  
}
```

- The **preg_match()** function searches a string for pattern, returning true if the pattern exists, and false otherwise.